

# PUTrack: Improved Underwater Object Tracking via Progressive Prompting

Qiuyang Zhang , Wei Song , *Member, IEEE*, Cong Liu, and Minghua Zhang

**Abstract**—Existing underwater tracking methods can be categorized into two paradigms: first, “enhance-then-track”—first enhancing the quality of the input image, then employing an open-air tracker; second, “track-then-process”—initially using an open-air tracker, followed by calibrating the prediction box. These methods that lack unified objectives among the modules impair tracking performance. To overcome this, we propose a novel end-to-end framework called prompting underwater tracking (PUTrack). It adapts the open-air tracker to the scenario-specific (underwater) tracking task by deploying a set of underwater prompts at the lateral side of an existing open-air tracker, and injecting the generated prompts layer-by-layer into the encoder. Experiments on various underwater tracking datasets demonstrate that the method significantly improves the underwater performance of the tracker by introducing only 0.6 M trainable parameters (0.4% of total parameters). Moreover, to drive the development of underwater tracking, we construct a high quality underwater tracking dataset that is manually annotated with 139 k frames, which exceeds the total number of frames in previous underwater tracking datasets. It provides 90 test sets rich in challenge properties and 200 training sets of diverse kinds.

**Index Terms**—Parameter efficient fine-tuning, underwater tracking, underwater tracking dataset, visual object tracking, visual prompt learning.

## I. INTRODUCTION

UNMANNED underwater vehicles are instrumental in a variety of underwater missions, including the exploration of underwater environments [1], the collection of biological information [2], and rescue operations [3]. Object tracking technology is an essential component that facilitates the successful completion of these missions. Over recent years, the advancement of deep neural networks and the availability of extensive tracking datasets have significantly enhanced the capabilities of open-air trackers. However, these trackers do not perform as well in underwater conditions. The unique challenges posed by

the underwater environment, such as poor lighting and altered color representation, result in image degradation and noise. Moreover, the distinct behavioral patterns of marine life and the limited availability of underwater datasets further impede the effectiveness of these tracking methods. Consequently, there is a pressing need to investigate robust underwater tracking methodologies that can elevate the underwater performance of open-air trackers.

Several studies have been conducted to improve the underwater tracking capability of open-air trackers. A general idea is gain enhancement by extracting more accurate features or improving target appearance adaptation. For example, Rout et al. [4] proposed an approach to adaptively select the Walsh–Hadamard kernels for the efficient extraction of color, edge, and texture. Wu et al. [5] introduced deep residual networks to enhance feature extraction of tracker and Lu et al. [6] proposed new modules to make the network adaptively fuse multiscale features. Other methods do not change the structure of the open-air tracker, but preprocess or postprocess the input and output of the model with the consideration of the underwater properties. For example, Li et al. [7] adopted the “enhance-then-track” paradigm [as shown in Fig. 1(a)], where underwater images are optimized using a separately trained image enhancement module before they are input to an open-air tracker. Li et al. [8] adopted the “track-then-process” paradigm [as shown in Fig. 1(b)], which calibrates the prediction box of the open-air tracker using a Kalman filter-based postprocessing module. It also uses a training set generated from the underwater detection dataset to retrain the open-air tracker. Despite improvements, current underwater tracking methods still have significant limitations, as follows.

- 1) Processing before or after tracking makes the tracker overly dependent on additional trained modules. Dividing training into two subprocesses is not conducive to building an end-to-end training architecture, and the training of the subprocesses may deviate from a unified goal.
- 2) Realistic underwater tracking datasets are scarce, and tracking samples generated from object detection datasets do not have interframe continuity, resulting in poorly trained trackers that suffer from overfitting and the inability to learn the continuous motion of the target.
- 3) Neglecting the intrinsic connection between the open-air tracker and the underwater tracker, and not exploiting the potential of the open-air tracker to be applied in underwater scenarios.

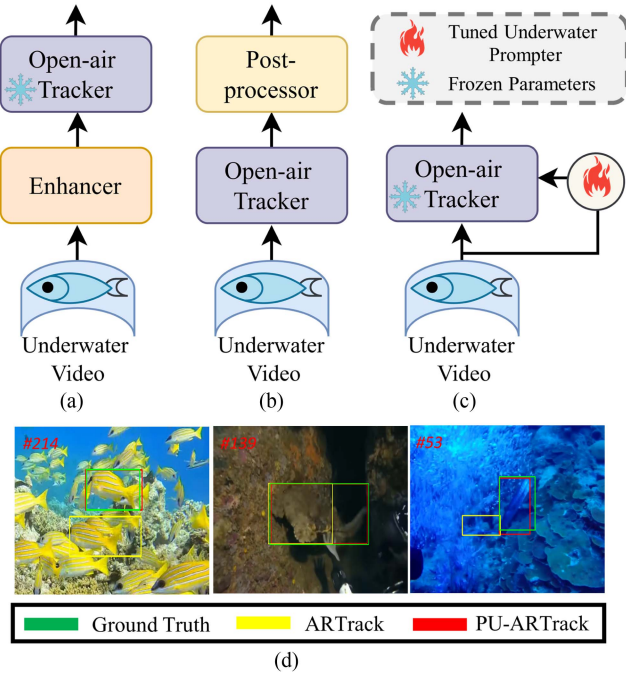
A research question remains: “How can we design an end-to-end underwater tracking architecture that leverages existing

Received 26 September 2024; revised 17 December 2024; accepted 24 January 2025. This work was supported by the National Natural Science Foundation of China under Grant 61972240. Paper no. TII-24-4997. (Corresponding author: Wei Song.)

The authors are with the School of Information, Shanghai Ocean University, Shanghai 201306, China (e-mail: m220951665@st.shou.edu.cn; wsong@shou.edu.cn; d230300088@st.shou.edu.cn; mhzhzhang@shou.edu.cn).

All resources are available at this <https://github.com/faicaiwawa/PUTrack>.

Digital Object Identifier 10.1109/TII.2025.3538056



**Fig. 1.** Illustration of different underwater tracking paradigms. (a) “enhance-then-track” paradigm. (b) “track-then-process” paradigm. (c) Proposed progressive PUTrack paradigm. (d) Three visualizations of the tracking results show that after deploying the PUTrack method for the open-air tracker ARTrack, the tracker’s performance in the three challenge scene of similar distractors, biomimicry, and LL are significantly improved.

open-air tracking models to overcome the current limitation of insufficient samples for underwater tracking?”

Recently, parameter-efficient fine-tuning methods have attracted much attention, which have been proposed from the field of natural language processing [9] and gradually extended to the field of computer vision [10]. Among them, prompt learning method are even more outstanding in the multimodal tracking domain [11], [12], [13] (e.g., RGB-Thermal tracking [11], RGB-Deep tracking [11], etc.). It enable the network to learn effective complementary model prompts by freezing the baseline model and training only a small number of introduced parameters. These works show favorable results and efficiency. Inspired by these works, we regard the construction of the underwater tracker as a prompt learning task. By injecting underwater prompts into the open-air tracker to achieve domain adaptation from open-air to underwater, as shown in Fig. 1(c). Specifically, we propose prompting underwater tracking (PUTrack), a progressive PUTrack framework that deploys a set of underwater prompters (UPTs) on the lateral side of currently popular vision transformer (ViT)-based trackers [14]. These prompters only require training on a few underwater tracking samples to learn underwater prompts and propagate them to all semantic layers of a pretrained open-air tracker. This method preserves the weights of the open-air tracker trained on a large-scale open-air dataset while simultaneously performing domain adaptation (underwater) effectively. We experiment the PUTrack framework on two differently-structured baseline trackers, OSTRack [15] and ARTrack [16], demonstrating that it performs comparable to or

better than full fine-tuning with only 0.6 M (0.4% of model parameters) of the introduced parameters for training.

In addition, in order to advance the development of underwater object tracking, we summarize the problems of past underwater tracking datasets and construct a new large-scale underwater tracking dataset, high quality underwater tracking (HUT290). The dataset is manually labeled with 290 sequences, about 139 k frames, of which 90 challenging sequences are used as the test set, and 200 scene-diverse sequences are used as the training set. The dataset contains rich challenge attribute scenarios and provides an evaluation toolkit.

The main contributions are summarized as follows.

- 1) We propose a novel underwater tracking framework called PUTrack. This method improves the performance of the open-air tracker in underwater scenes by mining underwater prompts and progressively propagating them to all semantic layers of the open-air tracker.
- 2) We propose a new large-scale underwater tracking dataset, HUT290, which is manual-labeled with a total of 139 k frames, divided into a training set and a test set. The dataset is substantially larger than its predecessors and contains diverse underwater organisms as well as tracking scenarios. Multiple challenge attribute are built into the dataset and an evaluation toolkit is provided.
- 3) Experiments on three underwater tracking benchmarks, UOT100, UTB180, and our HUT290-Test, validate the excellent performance of PUTrack. For example, deploying PUTrack on ARTrack requires only a fine-tuning of the introduced 0.6 M (0.4% of the ARTrack parameters) trainable parameters and improves the area under the curve (AUC) metrics by 4.1% and 4.8% on UTB180 and HUT290-TEST, respectively, with a performance better than full fine-tuning.

## II. PUTRACK METHOD

In this section, we provide a detailed description of proposed underwater tracking framework.

### A. Overall Architecture

Fig. 2 illustrates the overall workflow of PUTrack. A well-trained open-air tracker has excellent tracking capabilities in general scenarios, but lacks scenario-specific adaptation. To address this, we adjust only the weights of the prompt-related modules during training, rather than fine-tuning the entire model. Specifically, the template and search region go through patch embedding to generate foundation token flow  $\mathcal{H}^i, i \in 1, \dots, N$ . The token flow then passes through the UPT module to generate underwater prompts  $\mathcal{P}^i, i \in 1, \dots, N$ . Finally, the token flow and underwater prompts are gated, aggregated, and passed into the encoder layer. With this progressive propagation, the PUTrack framework retains the generalized tracking capability trained on the large-scale dataset [17], [18], [19], [20], avoiding overfitting on a smaller amount of underwater tracking datasets, while obtaining the scenario-specific tracking capability.

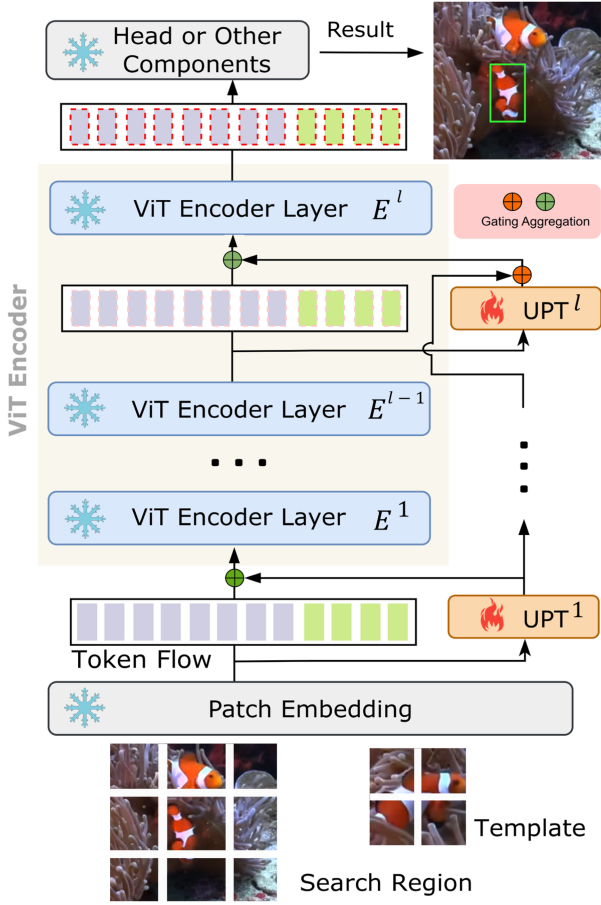


Fig. 2. Overview architecture of PUTRack. PUTRack is deployed on a ViT-based open-air tracking model. Initially, the template and search region are processed through the patch embedding layer, generating the corresponding tokens. These tokens are concatenated and fed into the model's backbone. Within the backbone, the token stream is input into the UPT, which produces refined underwater prompts. The network then combines these refined prompts with the underwater prompts from the previous layer, propagating the aggregated prompts to both the token flow and the next UPT. Finally, the prediction result is output through the prediction head or other relevant components of the tracker.

### B. Foundation of Open-Air Trackers

Open-air trackers offer established foundational capabilities for underwater tracking, including scene understanding, segmentation of front and rear views, target correlation, and comprehension of motion timing. In order to verify the generalizability and adaptability of PUTRack, we select two models with different architectures and widely referenced by researchers as baselines: OSTRack [15] and ARTrack [16]. OSTRack is one of the earliest proposed one-stream, on-stage trackers. It is a simple, effective and powerful baseline, and much subsequent works [11], [16], [38] have been carried out on its basis. ARTrack does not follow the architecture of previous object tracking models, but proposes a powerful autoregressive model that models tracking as a generation task of the target sequence.

We simplify the network structure of baseline into a ViT encoder and other components to facilitate the elaboration of our approach, as shown in Fig. 2.

The input of baseline tracker is a pair of images, namely, the template image  $Z \in \mathbb{R}^{3 \times H_z \times W_z}$  and the search region  $X \in \mathbb{R}^{3 \times H_x \times W_x}$ . They are first split and flattened into a series of patches  $Z_p \in \mathbb{R}^{N_z \times (3 \cdot P^2)}$  and  $X_p \in \mathbb{R}^{N_x \times (3 \cdot P^2)}$ , where  $P^2$  is the resolution of each patch, and  $N_z = H_z W_z / P^2$ ,  $N_x = H_x W_x / P^2$  are the number of patches of template and search region, respectively. Then, a trainable linear projection layer is used to project  $Z_p$  and  $X_p$  into a latent space with the added positional embedding  $E_{\text{pos}}$

$$\mathcal{H}_Z, \mathcal{H}_X = E_{\text{embedding}}(Z_p, X_p) + E_{\text{pos}}. \quad (1)$$

The template and search tokens are then concatenated to  $\mathcal{H}_{\text{base}}^0 = \text{Concat}(\mathcal{H}_Z, \mathcal{H}_X)$  and pass through an  $N$ -layer ViT encoder

$$\mathcal{H}^l = E^l(\mathcal{H}^{l-1}), \quad l = 1, 2, \dots, N \quad (2)$$

where  $E^l$  denotes the  $l$ th Encoder layer.

Finally, different trackers process the encoder's output differently to obtain the final prediction result

$$B = \phi(\mathcal{H}^N) \quad (3)$$

where  $\phi$  is a simple convolution-layered corner prediction head for OSTRack, and a complex altered transformer decoder for ARTrack.

### C. Underwater Prompter

The ViT-based open-air tracker demonstrates strong performance in open-air environments. It relies on the transformer's powerful long-range dependency capture and global information sensing capabilities. However, the performance in underwater scenarios is not satisfactory, and we analyze the reasons for this, as follows.

- 1) The behavioral patterns of underwater organisms are different from those of terrestrial organisms, and the targets to be tracked often appear in groups and are extremely similar in appearance, which makes it more difficult to discriminate similar distractors.
- 2) The mimicry strategy of underwater organisms makes them blend in with the surrounding environment, which makes it difficult to categorize the front and back view.
- 3) Underwater image imaging degradation, commonly such as blurring, dimness, and color bias, leads to the rising difficulty of capturing detail information.

Convolutional operation has excellent high-frequency information extraction ability and local feature capture ability, which can make up for the deficiency of the weak detail capture ability of the ViT very well.

Therefore, we design a kind of UPT based on the convolutional structure, which can extract the high-frequency information and detail representations in the token flow, and propagate them to all semantic layers by prompt injection. It is demonstrated through extensive experiments that this approach can well mitigate the abovementioned problems and introduces only 0.4% of the trainable parameters of the original model.

The detail design of UPT is shown in Fig. 3, consists of the following three main steps.



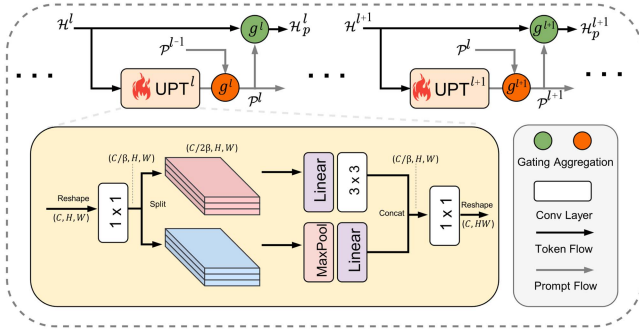


Fig. 3. Detailed structure of the proposed UPT module.

- 1) Projecting the input token flow to lower dimensional latent embedding.
- 2) Separating the features, extract high-frequency information and local feature representations in parallel.
- 3) Merging the features and project back to the original dimension to generate underwater prompts.

LORA [21] mentions that the key to fine-tuning in natural language pretraining models is low intrinsic rank. Inspired by this, we hypothesize that in visual tracking tasks, critical information obtained from a small number of low-rank features can generate more effective visual prompts. So we project the token flow to the low-dimensional space  $\frac{C}{\beta} \times H \times W$  to reduce the redundant features by

$$\mathcal{M} = f_1(\mathcal{H}^l) \quad (4)$$

where  $f_1(\cdot)$  is a  $1 \times 1$  convolutional layer, and we set  $\beta = 24$  in this article. Then,  $\mathcal{M}$  is divided into  $\mathcal{M}_1, \mathcal{M}_2 \in \mathbb{R}^{\frac{C}{\beta} \times H \times W}$  along the channel.  $\mathcal{M}_1$  is fed into a linear layer and a  $3 \times 3$  convolutional layer  $f_2$ , and  $\mathcal{M}_2$  is embedded with max-pooling and a linear layer

$$\begin{aligned} \mathcal{M}_3 &= f_2(\text{Linear}(\mathcal{M}_1)) \\ \mathcal{M}_4 &= \text{Linear}(\text{MaxPool}(\mathcal{M}_2)) \end{aligned} \quad (5)$$

where  $\mathcal{M}_3$  and  $\mathcal{M}_4$  are the outputs of two branches. Finally, we gate the UPT output of the previous layer with the UPT output of this layer, underwater prompts  $\mathcal{P}^l$  can be gained by

$$\begin{aligned} \mathcal{P}^l &= g^l \times f_3(\text{Concat}(\mathcal{M}_3, \mathcal{M}_4)) + (1 - g^l) \times \mathcal{P}^{l-1} \\ g^l &= \frac{1}{1 + e^{-\gamma^l}} \end{aligned} \quad (6)$$

where  $f_3$  is same as  $f_1$ ,  $\mathcal{P}^{l-1}$  is the prompt output by the previous UPT, and  $g^l$  is the gating aggregation weight generated through a sigmoid function, controlled by a parameter  $\gamma^l$ .  $\gamma$  is a learnable tensor of length 1, which we initialize to  $-10$ . This makes  $\gamma$  close to 0 after activation, meaning that the gate is close to closing. We want the network to extract a small amount of prompts from the previous layer and focus on the current layer at the beginning.

Subsequently,  $\mathcal{P}^l$  and  $\mathcal{H}^l$  perform gating aggregation

$$\begin{aligned} \mathcal{H}_p^l &= \mathcal{H}^l + t^l \times \mathcal{P}^l \\ t^l &= [t_1^l, t_2^l, \dots, t_M^l] \\ t_i^l &= \frac{1}{1 + e^{-\delta_i^l}}, \quad i = 1, 2, \dots, M \end{aligned} \quad (7)$$

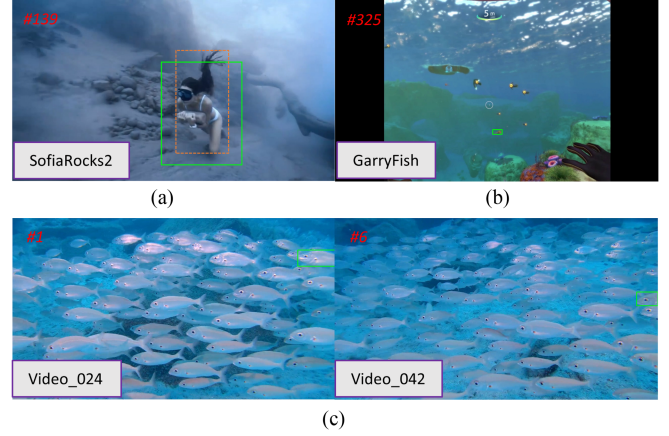


Fig. 4. Problems with previous underwater tracking datasets. In the lower left corner of each plot is the corresponding sequence name. (a) Green box is the label of the original dataset, and the orange box is a schematic representation of the exact labeling. (b) Cartoon underwater scenarios do not allow the tracker to learn realistic information about the underwater environment. (c) Datasets with similar data distributions will bias the learning of the tracker. (a) and (b) from UOT100, (c) from UTB180.

where  $M$  denotes the number of learned prompt tokens. The gating aggregation weight  $t^l \in \mathbb{R}^{1 \times M}$  is used to provide fine-grained control over the prompts injected into the token flow  $\mathcal{H}^l$ .  $\delta$  is a learnable tensor of the same length as the tokenflow. We consider prompts as important as the tokenflow, so we give them an initial value of 10. This makes each value in the tensor close to 1 after activation, and most prompts will be merged into the token flow at the beginning.

The proposed UPT is deployed at the lateral side of the ViT, and the outputs of the two neighboring UPTs are aggregated through gating and injected into the token flow in a fine-grained manner to achieve progressive prompting.

### III. HUT290 DATASET

This section analyzes the problems with the past underwater datasets and details the proposed HUT290 dataset.

#### A. Overview

Currently, most object tracking datasets focus on open scenes (such as streets, indoors, and natural environments) and lack underwater scenarios, leading to poor performance in underwater environments. Constructing an underwater tracking dataset can fill this gap by providing an objective and fair benchmark for underwater object tracking algorithms. Researchers can use this dataset to develop and verify algorithms in a consistent environment, thereby advancing the field of underwater object tracking. Improved accuracy in underwater tracking models can also promote marine exploration, ecological monitoring, and marine engineering.

The existing underwater datasets UOT100 [22] and UTB180 [23] promote the development of underwater tracking, but there are some shortcomings. As shown in Fig. 4(a) and (b), the UOT100 is not accurately labeled and contains a large number of artificially synthesized underwater videos (such as



TABLE I  
COMPARISON OF EXISTING OPEN-AIR TRACKING DATASETS AND UNDERWATER (UWT) TRACKING DATASETS

|          | Datasets         | Published  | Total |       | Test |       | Train |       | Properties |       |      |      |
|----------|------------------|------------|-------|-------|------|-------|-------|-------|------------|-------|------|------|
|          |                  |            | Seq.  | Boxes | Seq. | Boxes | Seq.  | Boxes | Min        | Max   | Avg  | F.R. |
| Open-Air | OTB2015 [24]     | TPAMI2015  | 100   | 59k   | 100  | 59k   | -     | -     | 71         | 3872  | 598  | 30   |
|          | UAV123 [25]      | ECCV2016   | 123   | 113k  | 123  | 113k  | -     | -     | 109        | 3085  | 1247 | 30   |
|          | NFS [26]         | ICCV2017   | 100   | 383k  | 100  | 383k  | -     | -     | 900        | 37440 | 4260 | 30   |
|          | VOT2018 [27]     | ICCV W2018 | 60    | 21k   | 60   | 21k   | -     | -     | 41         | 1500  | 356  | 30   |
|          | *OxUvA [28]      | ECCV2018   | 366   | 51k   | 200  | 18k   | 166   | 33k   | 900        | 37440 | 4260 | 30   |
|          | LaSOT [17]       | CVPR2019   | 1.4k  | 3.3M  | 280  | 685k  | 1.1k  | 2.8M  | 1000       | 11397 | 2506 | 30   |
|          | TrackingNet [19] | ECCV2018   | 31k   | 14M   | 511  | 226k  | 30k   | 14M   | 96         | 2368  | 471  | 30   |
|          | GOT-10K [18]     | TPAMI2019  | 10k   | 1.5M  | 420  | 56k   | 9.34k | 1.4M  | 29         | 1418  | 149  | 10   |
|          | TNL2K [29]       | CVPR2021   | 2k    | 1.2M  | 700  | 400k  | 1.3k  | 800k  | 21         | 18488 | 622  | 30   |
| UWT      | UOT100 [22]      | JOE2022    | 104   | 73k   | 104  | 73k   | -     | -     | 264        | 1764  | 702  | 30   |
|          | UTB180 [23]      | ACCV2022   | 180   | 60k   | 180  | 60k   | -     | -     | 40         | 1226  | 338  | 30   |
|          | HUT290           | OURS       | 290   | 139k  | 90   | 45k   | 200   | 95k   | 146        | 1956  | 480  | 30   |

\* indicates that this dataset is sparsely annotated at a frequency of 1 HZ. Seq. represents the number of sequences. Properties describes the number of sequence frames and the frame rate.

games, animations). As a result, the usage of this dataset as a training set poses challenges for the tracking model to effectively learn authentic underwater tracking circumstances. Similarly, when used as a test set, it becomes problematic to appropriately assess the performance of the tracker.

The UTB180 addresses the problem of imprecise labeling of the UOT100, and provides a collection of real underwater video sequences that are labeled with greater precision. Nevertheless, the dataset is limited in terms of the number of frames, and there are sequences with highly similar data distributions [as shown in Fig. 4(c)], which can bias the learning of the model.

The proposed HUT290 consists of 290 real underwater videos, all of which are manually labeled frame-by-frame to provide 139 K precise bounding boxes. A total of 90 challenging video sequences are used as a test set to comprehensively evaluate trackers' performance in underwater tracking tasks, and another 200 high-quality sequences are used as a training set to provide high-quality learning materials for underwater tracking models. Compared to previous underwater tracking datasets, as shown in Table I, HUT290 is larger, more accurately labeled, and has a wider and more balanced data distribution.

## B. Collection and Annotation

HUT290 is sourced multiple publicly available online sources such as YouTube, Pixel, and BiliBili. We also select sequences from our lab's video library to increase the diversity of video scenes and target categories. The maximum, minimum, and average frame resolution of the sequences are  $2056 \times 1156$  pixels,  $480 \times 270$  pixels, and  $1617 \times 915$  pixels at 30 frames per second. As shown in Fig. 5, we collect the types of tracked objects and the color cast of the images in HUT290. The types of tracked objects are dominated by various types of fish, followed by humans. We classify other unidentifiable creatures as "other." In terms of color cast, most images are colorless, followed by blue and green, and finally purple.

We use the format  $[X_{\text{top-left}}, Y_{\text{top-left}}, W, H]$  to label each frame, being consistent with [17], [19]. When the target is out of the view, the labeling of the target disappeared frame remains at the position of the previous frame before the target disappeared.

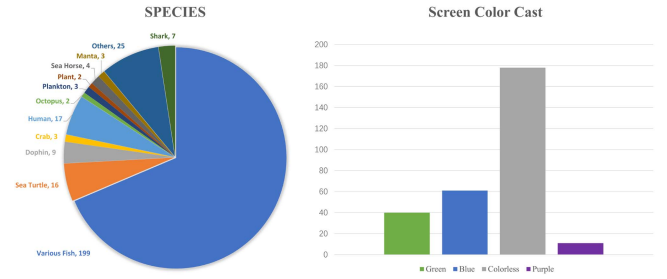


Fig. 5. Species distribution and color cast statistics for the HUT290.

## C. Challenge Attributes

Challenge attributes are key to evaluating a tracker. Scrutinizing the performance of a tracker on videos with different challenge attributes can help to discern the strengths and weaknesses of the tracker facing various scenes. We define the following 11 challenge attributes for underwater tracking:

- 1) *Angle change (AC)*: The camera angle has changed dramatically.
- 2) *Deformation (DEF)*: The target is severely deformed.
- 3) *Rotation (ROT)*: The target takes a significant ROT.
- 4) *Fast movement (FM)*: The target is moving fast.
- 5) *Similar object (SO)*: SO interference.
- 6) *Low resolution (LR)*: LR image.
- 7) *Obscured (OBS)*: Target OBS.
- 8) *Background interference (BI)*: BI, which can also be called mimicry interference.
- 9) *Out-of-view (OV)*: The target moves out of the camera's field of view and then reappears.
- 10) *Turbid water (TW)*: Turbid water.
- 11) *Low light (LL)*: Dim lighting.

Each test sequence contains at least two challenges. We refer to the approach in [17] to mark the challenge attributes of each test sequence.

If a particular challenge attribute occurs in a sequence, the corresponding challenge item is marked as 1; otherwise, it is marked as 0.

TABLE II  
STATE-OF-THE-ART COMPARISONS ON THREE UNDERWATER TRACKING BENCHMARKS

| Methods          | Published  | UOT100                |                       |                       | UTB180                |                       |                       | HUT290                |                       |                       |
|------------------|------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
|                  |            | AUC(↑)                | P(↑)                  | $P_{Norm}$ (↑)        | AUC(↑)                | P(↑)                  | $P_{Norm}$ (↑)        | AUC(↑)                | P(↑)                  | $P_{Norm}$ (↑)        |
| KeepTrack [30]   | ICCV2021   | 55.6                  | 43.9                  | 65.1                  | 61.6                  | 52.6                  | 79.9                  | 42.0                  | 30.6                  | 47.9                  |
| STARK-ST50 [31]  | ICCV2021   | 65.9                  | 57.5                  | 80.6                  | 54.7                  | 46.5                  | 63.0                  | 42.2                  | 33.0                  | 47.0                  |
| TransT [32]      | CVPR2021   | 63.7                  | 56.2                  | 80.4                  | 57.1                  | 50.5                  | 66.2                  | 41.1                  | 33.3                  | 46.6                  |
| ToMP [33]        | CVPR2022   | 61.0                  | 53.7                  | 70.9                  | 66.4                  | 58.2                  | 82.7                  | 47.4                  | 37.9                  | 54.2                  |
| AiaTrack [34]    | ECCV2022   | 65.6                  | 58.8                  | 84.2                  | 62.3                  | 53.1                  | 72.7                  | 49.5                  | 39.8                  | 56.6                  |
| SimTrack [35]    | ECCV2022   | 64.2                  | 56.7                  | 81.0                  | 56.6                  | 49.0                  | 64.1                  | 42.1                  | 32.3                  | 46.6                  |
| MIXFormer [37]   | CVPR2022   | 66.3                  | 60.3                  | 84.0                  | 57.4                  | 50.6                  | 65.9                  | 45.1                  | 37.6                  | 50.6                  |
| OStTrack [15]    | ECCV2022   | 67.5                  | 62.9                  | 85.2                  | 64.8                  | 59.1                  | 75.1                  | 48.5                  | 42.1                  | 54.8                  |
| MIXFormerV2 [36] | NIPS2023   | 66.5                  | 59.0                  | 82.4                  | 55.6                  | 47.9                  | 63.3                  | 43.5                  | 34.3                  | 48.0                  |
| *UOStTrack [8]   | Ocean Eng. | 68.2                  | 63.2                  | 85.6                  | 66.7                  | 60.3                  | 77.0                  | 50.6                  | 43.4                  | 56.7                  |
| SeqTrack [38]    | CVPR2023   | 66.6                  | 62.7                  | 85.7                  | 60.8                  | 55.2                  | 70.4                  | 46.4                  | 40.4                  | 53.9                  |
| GRM [39]         | CVPR2023   | 67.2                  | 62.1                  | 85.0                  | 64.7                  | 59.2                  | 74.2                  | 49.8                  | 43.2                  | 55.9                  |
| ARTrack [16]     | CVPR2023   | 66.1                  | 62.4                  | 86.2                  | 69.0                  | 65.1                  | 78.6                  | 54.5                  | 49.9                  | 61.0                  |
| ODTrack [40]     | AAAI2024   | 68.5                  | 66.0                  | 89.4                  | 70.1                  | 64.6                  | 82.0                  | 53.8                  | 49.5                  | 60.9                  |
| *PU-OStTrack     | OURS       | 68.3 <sub>(0.8)</sub> | 64.2 <sub>(1.3)</sub> | 85.0 <sub>(0.2)</sub> | 67.0 <sub>(2.1)</sub> | 61.6 <sub>(2.5)</sub> | 77.0 <sub>(1.9)</sub> | 50.7 <sub>(2.2)</sub> | 44.5 <sub>(2.4)</sub> | 57.3 <sub>(2.5)</sub> |
| *PU-ARTrack      |            | 69.0 <sub>(2.9)</sub> | 65.2 <sub>(2.8)</sub> | 87.6 <sub>(1.4)</sub> | 73.1 <sub>(4.1)</sub> | 70.5 <sub>(5.4)</sub> | 82.7 <sub>(4.1)</sub> | 59.3 <sub>(4.8)</sub> | 55.2 <sub>(5.3)</sub> | 66.3 <sub>(5.3)</sub> |

The trackers are compared in terms of Precision ( $P$ ), normalized precision ( $P_{Norm}$ ), area under the curve (AUC). Results are reported in percentage (%). The best three results are shown in red, blue and green font. The values in brackets represent the lift of our methods compared to their respective baselines. \* stands for underwater trackers.

In addition, we offer a MATLAB-based evaluation toolkit to assist researchers when evaluating their models.

#### IV. EXPERIMENT

In this section, we first describe the implementation details of PU-ARTrack and PU-OStTrack, followed by comparison and exploration experiments.

##### A. Implementation Details

*Training and testing equipment:* Our trackers are implemented in Python 3.8 and PyTorch 1.9. The models are trained on a server with two Nvidia Titan RTX GPUs, each with 24 GB of memory. Test device using RTX 3060 GPU with 6 GB of memory and Ryzen 6900HX CPU.

*Training sets:* In Table II, the open-air trackers used for comparison are trained using four open-air datasets: LaSOT-train [17], GOT-10K-train [18], TrackingNet-train [19], and COCO17 [20]. We use the pretrained models provided by the authors for testing. UOStTrack uses the abovementioned four training sets and adds the RUOD [41] underwater detection dataset and the self-made dataset FishExtend. Our PU-ARTrack and PU-OStTrack use LaSOT-train, GOT-10K-train, and HUT290-train (the sample ratio for each epoch is 1:1:1) for prompt-based fine-tuning based on the corresponding pretrained open-air trackers. These two open-air datasets are mixed into the training set to prevent the model from overfitting to the relatively small underwater dataset.

*Prompt fine-tuning:* Pretrained ARTrack and OStTrack are selected as baseline models. During fine-tuning, the baseline models are frozen, and only prompt-related parameters are tuned. For PU-OStTrack, we tune the prompt module for 15 epochs, reducing the learning rate to one-tenth after the tenth epoch. Other settings are the same as OStTrack. For PU-ARTrack, we tune the prompt module for 20 epochs, where the CE loss weight is set to 2 and the learning rate is set to  $8e-5$  without decreasing the learning rate. Other settings consistent with the second

stage (ARTrack\_seq) training strategy of ARTrack, and our code implementation is also based on the second stage (ARTrack\_seq) of ARTrack.

##### B. Comparison With the State-of-the-Art Trackers

We compare PU-ARTrack and PU-OStTrack with other state-of-the-art (SOTA) trackers on three benchmark datasets, UOT100, UTB180, and HUT290-Test, as shown in Table II. The comparison strategy here refers to previous related works [8], [42], [43]. The comparison aims to illustrate the improvement of our fine-tuning method over the baseline, as well as to provide the performance of various open-air trackers in underwater scenes and facilitate future research.

The purpose of the comparison is to demonstrate the improvement of our fine-tuning method for baseline trackers, as well as to demonstrate the performance of various open-air trackers in underwater scenes to promote related developments.

Precision( $P$ ), normalized precision( $P_{Norm}$ ), and AUC are used as evaluation metrics.  $P$  is the percentage of frames with estimated positions within a threshold distance of ground-truth centers.  $P_{Norm}$  is calculated by dividing  $P$  by the average object size, providing a measure of accuracy that is normalized across different object scales. AUC is area under the curve of the success rate plot.

*Evaluation on UOT100:* The AUC metric of PU-ARTrack on UOT100 achieved the optimum and  $P$  and  $P_{Norm}$  achieved the second best. Compared to the baseline ARTrack, the AUC value improved by 2.9%, and  $P$  and  $P_{Norm}$  improved by 2.8% and 1.4%, respectively. The PU-OStTrack improved the AUC by 0.8%,  $P$  by 1.3%, and  $P_{Norm}$  by 0.2% compared to the baseline OStTrack. We consider that this may be due to the fact that the GT labeling of UOT100 is too loose, and the bounding box predicted by the model after training is more close to the actual target, then the model's prediction box will be shifted larger relative to the GT. This will lead to a decrease in  $P_{Norm}$ . *Evaluation on UTB180:* The UTB180 annotation is more accurate than

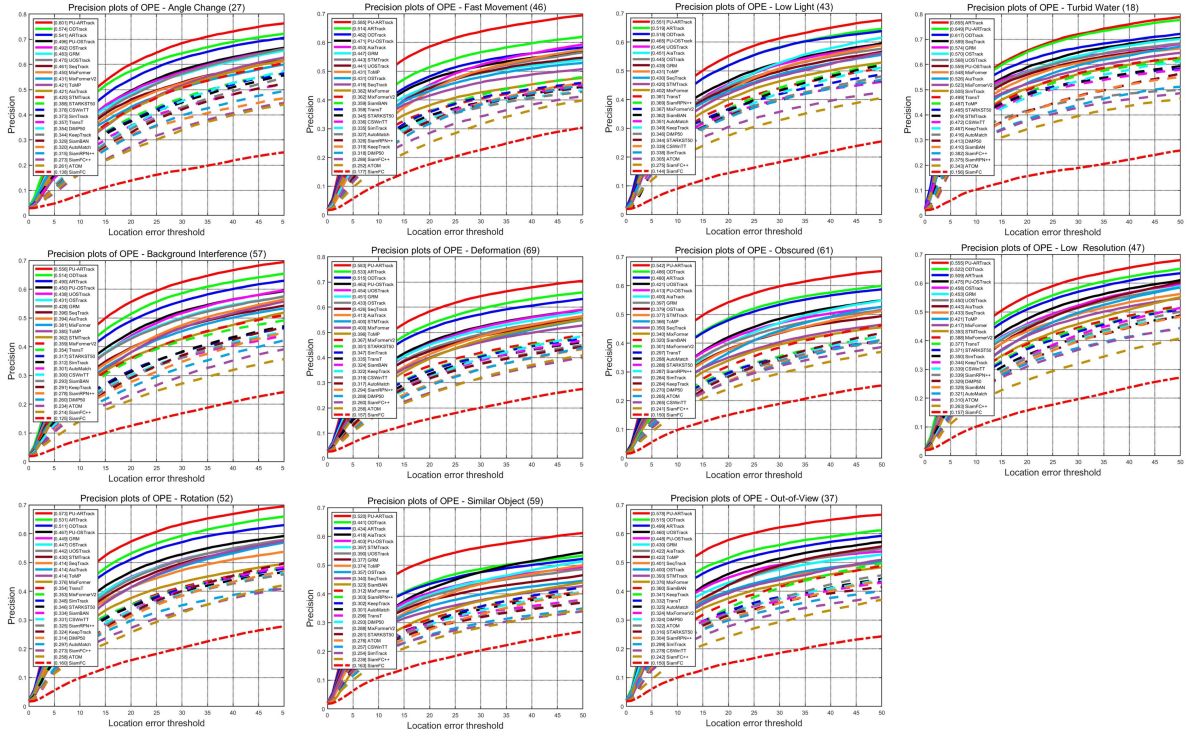


Fig. 6. Challenge attribute score comparison of state-of-the-art trackers on HUT290-TEST. The number after the challenge name represents the number of sequences containing that challenge. For example, AC (27) indicates that there are 27 sequences in the test set that include AC scenarios.

that of the UOT100, and tests on UTB180 more accurately reflect the model's performance. PU-ARTrack achieves the best performance in AUC,  $P$ , and  $P_{\text{Norm}}$ , which are 3%, 5.4%, and 0.7% ahead of the suboptimal values, respectively. Compared with the baseline ARTrack the improvement is more obvious, especially AUC and  $P$  improved by 4.1%, and 5.4%, respectively. PU-OSTrack shows an improvement of 2.1%, 2.5%, and 1.9% in AUC,  $P$ , and  $P_{\text{Norm}}$ , respectively, compared to OSTrack.

**Evaluation on HUT290:** Overall, the performance of all trackers on HUT290 is a little less accurate compared to the previous two datasets, which suggests that our proposed test set is more challenging with higher upper bounds compared to the previous test set. PU-ARTrack still achieves the optimal performance in AUC,  $P$ , and  $P_{\text{Norm}}$ . Compared to the suboptimal ARTrack, it is ahead by 4.8%, 5.3%, and 5.3%, respectively. PU-OSTrack improves by 2.2%, 2.4%, and 2.5% in AUC,  $P$ , and  $P_{\text{Norm}}$ , respectively, compared to OSTrack.

HUT290 is rich in challenge attributes. We evaluate the tracking precision of all the comparison trackers for different challenges and plot Fig. 6. It can be intuitively seen that PU-ARTrack achieves the optimum among the ten challenges, while it achieves the suboptimum in the challenge scenario of *TW*. PU-OSTrack outperforms the baseline OSTrack for the ten challenge scenarios, while there is a slight decrease in performance for the *TW* challenge.

### C. Exploration Studies

We further explore the characteristics of PUTrack on underwater benchmarks.

TABLE III  
ABLATION STUDIES ON THREE BENCHMARKS

| Methods     | Params <sup>†</sup> | UOT100      |             | UTB180      |             | HUT290      |             |
|-------------|---------------------|-------------|-------------|-------------|-------------|-------------|-------------|
|             |                     | AUC         | P           | AUC         | P           | AUC         | P           |
| ARTrack     | -                   | 66.1        | 62.4        | 69.0        | 65.1        | 54.5        | 49.9        |
| FFT-ARTrack | 173.1M(100%)        | 69.1        | 65.3        | 70.5        | 67.7        | 57.2        | 53.1        |
| PU-ARTrack  | 0.6M(0.4%)          | 69.0        | 65.2        | <b>73.1</b> | <b>70.5</b> | <b>59.3</b> | <b>55.2</b> |
| # + Adapter | 2.98M(1.7%)         | <b>70.3</b> | <b>65.7</b> | 72.9        | 70.5        | 57.0        | 52.3        |
| # + P-Token | 0.6M(0.4%)          | 68.8        | 64.3        | <b>73.1</b> | 70.4        | 57.9        | 53.9        |

Params<sup>†</sup> denotes the number of trainable parameters and its proportion of the overall model parameters noted in parentheses. Results are reported in percentage (%) with the best in red. # represent PU-ARTrack. FFT denotes full fine-tuning.

**Power of underwater prompt:** To investigate the benefits of underwater prompts, we compare ARTrack, PU-ARTrack, and FFT-ARTrack with HUT290-Train. The results are shown in Table III. Compared with the baseline ARTrack, the FFT-ARTrack performance is significantly improved, with 3%, 1.5%, and 2.7% AUC improvement on UOT100, UTB180, and HUT290, respectively. Comparing fully fine-tuning (FFT) and Prompt training, PU-Track can achieve a better performance than FFT with only a small number of parameters introduced by training, e.g., 2.1% AUC, and 2.1%  $P$  on HUT290, 2.6% AUC, and 2.8%  $P$  on UTB180.

As in the last two rows of Table III, we also attempt to deploy UPT along with other popular fine-tuning paradigms, such as adapter and prompt token, to investigate whether better fine-tuning performance can be achieved. For the Adapter, we refer to [9] to add the parallel adapters in the attention module and feed forward module of each encoder layer. We hope to use



**TABLE IV**  
PUTRACK INFERENCE AND TRAINING EFFICIENCY ANALYSIS

| # Inference  | Parameters(M) | FLOPs(G)      | Speed(fps) |
|--------------|---------------|---------------|------------|
| OTrack [15]  | 92.1          | 58.2          | 58         |
| ARTrack [16] | 173.1         | 79.3          | 21         |
| PU-OTrack    | 92.7          | 60.0          | 51         |
| PU-ARTrack   | 173.7         | 81.1          | 17         |
| # Training   | Epochs        | Peak VRAM(MB) |            |
| FFT-ARTrack  | 75            | 20577         |            |
| PU-ARTrack   | 20            | 13634         |            |

We employ the formula  $FLOPs = 2 \times MACs$  to calculate the FLOPs. FFT denotes full fine-tuning.

the adapter to further refine the prompted token flow in the way of residual linkage. After adding the adapter to PU-ARTrack, the performance of PU-ARTrack improves slightly on UOT100, but decreases on the other two benchmarks. We speculate that this may be due to the fact that the adapter learns too much about a certain data distribution characteristic that is more favorable on UOT100. For the prompt token, we refer to [10] to concatenate a set of trainable tokens of length 10 in the token flow of each encoder layer, and the trainable tokens of the previous layer will be retained to the next layer. We hope that this approach can be used with underwater prompts to positively affect attention. However, as shown in the experimental results, this approach does not result in a significant performance improvement, but rather a significant decrease. We speculate that this may be because the newly added prompt token plays a side effect on attention, which in turn affects the prediction results.

Although these two options are not performing well at this stage, we believe that this could be a research direction for future work.

*Efficiency analysis:* In Table IV, we analyze PUTrack in terms of both inference and training. On the inference side, we test the average speed of the compared trackers on three underwater datasets. The computation of this speed inference includes pre- and postprocessing time, such as search region cropping and padding. After deploying the PUTrack framework, the model's FLOPs increased by 1.8 G and the parameters increased by 0.6 M. This results in PU-OTrack and PU-ARTrack being 7 fps and 4 fps lower compared to their baselines. On the training side, we compare the occupancy of video random access memory (VRAM) between full fine-tuning and our prompt fine-tuning. Since our prompt tuning only requires updating the gradients of a small number of parameters, a large amount of intermediate activation values that do not participate in gradient computation can be released during forward and backward propagation, resulting in a 34% reduction in peak VRAM usage. In addition, it takes 75 epochs for full fine-tuning to achieve the optimum, while our prompt tuning only requires 20 epochs. This saves about 89 h (on our training device). Overall, the deployment of PUTrack introduces a small number of parameters, but greatly improves training efficiency while also achieving better model accuracy.

*Feature and prompt gate:* To explore the benefits of the two gate designs, we conduct ablation experiments, as shown in

**TABLE V**  
ABLATION STUDIES OF TOKEN GATE AND PROMPT GATE

| # | Methods    | Gate  |        | UOT100 |      | UTB180 |      | HUT290 |      |
|---|------------|-------|--------|--------|------|--------|------|--------|------|
|   |            | Token | Prompt | AUC    | P    | AUC    | P    | AUC    | P    |
| 1 | PU-ARTrack | ✗     | ✗      | 68.3   | 64.4 | 71.3   | 68.4 | 58.5   | 54.3 |
| 2 | PU-ARTrack | ✓     | ✗      | 69.0   | 64.9 | 72.8   | 70.1 | 59.0   | 54.6 |
| 3 | PU-ARTrack | ✓     | ✓      | 69.0   | 65.2 | 73.1   | 70.5 | 59.3   | 55.2 |

Results are reported in percentage (%) with the best in red.

**TABLE VI**  
ABLATION STUDY WITH DIFFERENT NUMBERS OF SPATIAL PROJECTION DIMENSION

| $\beta$ | UOT100 |      |                   | UTB180 |      |                   | HUT290 |      |                   |
|---------|--------|------|-------------------|--------|------|-------------------|--------|------|-------------------|
|         | AUC    | P    | P <sub>Norm</sub> | AUC    | P    | P <sub>Norm</sub> | AUC    | P    | P <sub>Norm</sub> |
| 96      | 67.8   | 63.9 | 85.4              | 71.6   | 68.8 | 81.3              | 56.1   | 51.4 | 62.2              |
| 48      | 68.6   | 64.0 | 86.0              | 72.0   | 69.3 | 81.9              | 57.0   | 52.2 | 62.8              |
| 32      | 68.2   | 64.6 | 85.6              | 72.5   | 69.7 | 82.4              | 57.9   | 53.9 | 64.4              |
| 24      | 69.0   | 65.2 | 87.6              | 73.1   | 70.5 | 82.7              | 59.3   | 55.2 | 66.3              |
| 16      | 69.3   | 65.6 | 87.4              | 72.9   | 70.6 | 82.7              | 58.7   | 54.8 | 65.6              |

The best results are shown in red font. Results are reported in percentage (%).

Table V. Prompt gate and token gate correspond to (5) and (7), respectively. Model #1 removes the token gate and prompt gate from the default PU-ARTrack, while retaining the UPT. Models #2 and #3 sequentially add the two gates back for comparison. Other training settings remain the same as the default configuration. The improvement of #1 relative to #2 is most obvious on UTB180, with an AUC increase of 1.5% and a  $P$  value increase of 1.7%. The improvement of #3 relative to #2 on UOT180 is not obvious, AUC remains unchanged, and the  $P$  value rises by 0.3%. On HUT290, AUC rises by 0.3% and  $P$  rises by 0.6%. Overall, the combination of the two gate mechanisms brings a certain improvement to the model, which is most significant on UTB180. This shows that the gating mechanism has the expected effect on the inflow of effective prompts.

*Spatial projection dimensions:* We explore the effect of the number of projection dimensions  $\frac{C}{\beta}$  in UPT, as shown in Table VI. When  $\beta = 24$ , PU-ARTrack performs best on the UTB180 and HUT290 benchmarks. On the UOT100 benchmark,  $\beta = 16$  outperforms  $\beta = 24$  by 0.3% in AUC and 0.4% in  $P$ . As can be seen, the results become relatively smooth from 24 onwards, and given the goal of an efficient module, we chose  $\beta = 24$  as the default setting.

#### D. Visualization Analysis

*Visualization of tracking results:* To provide a more visual demonstration of tracking performance, we select some challenging sequences from the HUT290 benchmark for visual comparison with other state-of-the-art trackers, as shown in Fig. 7.

*Fish157 and Fish48* contain three challenge attributes of SO interference, LL, and FM. In fish157, the compared SOTA trackers lose the target at frame 91 and the predictions drift to similar surrounding fish. The PU-ARTrack, on the other hand, correctly discriminates the target to be tracked and accurately predicts its scale throughout.

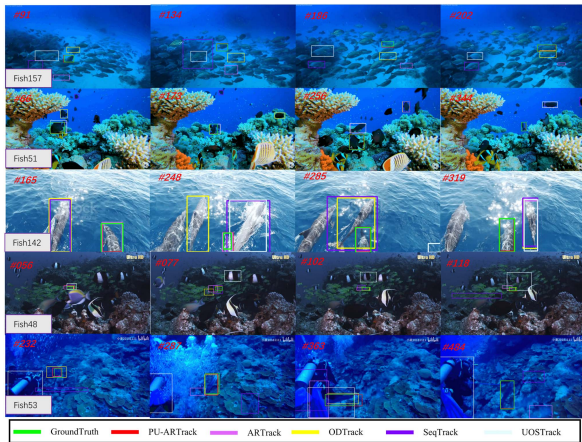


Fig. 7. Visualization of tracking results in representative underwater scenarios.

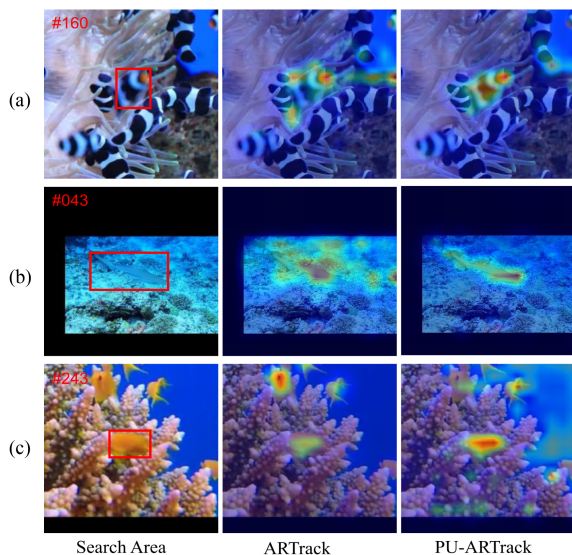


Fig. 8. Visualization of attention weights for the PU-ARTrack and ARTrack encoders. Blue → green → red indicates a low to high attention weight. The red rectangle is the ground truth.

In *Fish51*, the sequence contains two challenging attributes of SO interference and severe DEF. From frame 66 to 173, PU-Track and UOTrack can accurately predict the location of the target, while the other trackers lose the target. From frame 256 onwards, UOTrack’s prediction also drifts, while PU-ARTrack steadily tracks the target.

In *Fish142*, the sequence contains three challenge attributes of SO interference, OV and FM. PU-ARTrack follows the target stably throughout the process, while other methods have difficulty in distinguishing the target to be tracked when the three challenge attributes are concurrent, and gradually drift to the interfering object during the tracking process.

In *Fish53*, the sequence contain the challenges of BI, extremely low brightness, and severe color cast. ARTrack and other SOTA trackers have difficulty capturing the target. However, the prompted PU-ARTrack can robustly distinguish between the foreground and background.

*Visualization of the encoder attention:* As shown in Fig. 8, we take the attention weights of the last encoder layer for visualization. Specifically, we visualize the attention weights of the center token of the template to the search area tokens as a heat map. The three scenes (a), (b), and (c) correspond to three difficulty challenges: SO, BI, and OBS. In sequence (a), after deploying PUTrack, the model can focus more on the tracked object and not be distracted by SOs. In sequence (b), underwater prompting allows the model to segment the foreground and background more accurately. In sequence (c), even if the highly recognizable part of the tracked object (the head) is occluded, the location of the object can still be inferred robustly through prompting.

## V. CONCLUSION

In this article, we introduce an end-to-end underwater tracking framework called PUTrack. It adapts a pretrained open-air tracker to underwater environments by mining underwater prompts from token flow and propagating them across all semantic layers of the encoder. Extensive experiments demonstrate that underwater tracking accuracy can be significantly improved by introducing only 0.4% of the original model’s trainable parameters (0.6 M). In addition, to advance underwater tracking research, we construct a new large underwater tracking dataset called HUT290. The dataset is manual-labeled with 139 k bounding boxes, far exceeding previously available underwater datasets. The dataset is divided into two parts: a training set that has excellent tracking scenarios for the tracker to learn, and a test set that offers rich challenge attributes for the tracker to assess. This will enhance the progress of study in the realm of underwater exploration.

Despite the promising results of PUTrack, it faces the same problem as all parameter-efficient fine-tuning methods: when fine-tuning a small number of introduced parameters, the entire baseline model still needs to be loaded into VRAM for calculation, which places high demands on the hardware.

In addition, although the HUT290 contains a wealth of underwater scenes, there are still rare scenes that are not covered, such as deep sea tracking videos and tracking videos in various water qualities. We will continue to collect more challenging scenes and target types in our future work.

## REFERENCES

- [1] I. Jawhar, N. Mohamed, J. Al-Jaroodi, and S. Zhang, “An architecture for using autonomous underwater vehicles in wireless sensor networks for underwater pipeline monitoring,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 3, pp. 1329–1340, Mar. 2019.
- [2] J. Yuan, Z. Cai, and W. Cao, “A novel underwater detection method for ambiguous object finding via distraction mining,” *IEEE Trans. Ind. Informat.*, vol. 20, no. 7, pp. 9215–9224, Jul. 2024.
- [3] Y. R. Petillot, G. Antonelli, G. Casalino, and F. Ferreira, “Underwater robots: From remotely operated vehicles to intervention-autonomous underwater vehicles,” *IEEE Robot. Autom. Mag.*, vol. 26, no. 2, pp. 94–101, Jun. 2019.
- [4] D. K. Rout, B. N. Subudhi, T. Veerakumar, and S. Chaudhury, “Walsh–Hadamard-kernel-Based features in particle filter framework for underwater object tracking,” *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 5712–5722, Sep. 2020.



- [5] X. Wu et al., "A hybrid excitation model based lightweight siamese network for underwater vehicle object tracking missions," *J. Mar. Sci. Eng.*, vol. 11, no. 6, 2023, Art. no. 1127.
- [6] Y. Lu et al., "Multi-scale underwater object tracking by adaptive feature fusion," *Proc. SPIE*, vol. 11884, 2021, pp. 346–357.
- [7] Y. Li et al., "UStark: Underwater image domain-adaptive tracker based on stark," *J. Electron. Imag.*, vol. 31, pp. 53012–53012, 2022.
- [8] Y. Li et al., "Underwater object tracker: UOSTrack for marine organism grasping of underwater vehicles," *Ocean Eng.*, vol. 285, 2023, Art. no. 115449.
- [9] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–15.
- [10] C. Xu et al., "Progressive visual prompt learning with contrastive feature re-formation," in *Proc. Int. J. Comput. Vis.*, 2024, pp. 1–16.
- [11] J. Zhu, S. Lai, X. Chen, D. Wang, and H. Lu, "Visual prompt multi-modal tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 9516–9526.
- [12] J. Zhu et al., "DCPT: Darkness clue-prompted tracking in nighttime UAVs," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 7381–7388.
- [13] M. Zhang, Q. Zhang, W. Song, D. Huang, and Q. He, "PromptVT: Prompting for efficient and accurate visual tracking," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 8, pp. 7373–7385, Aug. 2024, doi: [10.1109/TCSVT.2024.3376582](https://doi.org/10.1109/TCSVT.2024.3376582).
- [14] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–21.
- [15] B. Ye et al., "Joint feature learning and relation modeling for tracking: A one-stream framework," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 341–357.
- [16] X. Wei, Y. Bai, Y. Zheng, D. Shi, and Y. Gong, "Autoregressive visual tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 9697–9706.
- [17] H. Fan et al., "LaSOT: A high-quality benchmark for large-scale single object tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 5374–5383.
- [18] L. Huang, X. Zhao, and K. Huang, "GOT-10 k: A large high-diversity benchmark for generic object tracking in the wild," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 5, pp. 1562–1577, May 2021.
- [19] M. Muller et al., "TrackingNet: A large-scale dataset and benchmark for object tracking in the wild," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 300–317.
- [20] T. Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
- [21] E. J. Hu et al., "LORA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–21.
- [22] K. Panetta, L. Kezebou, V. Oludare, and S. Agaian, "Comprehensive underwater object tracking benchmark dataset and underwater image enhancement with GAN," *IEEE J. Ocean. Eng.*, vol. 47, no. 1, pp. 59–75, Jan. 2022.
- [23] B. Alawode et al., "Utb180: A high-quality benchmark for underwater tracking," in *Proc. Asian Conf. Comput. Vis.*, 2022, pp. 3326–3342.
- [24] Y. Wu, J. Lim, and M. -H. Yang, "Object tracking benchmark," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 9, pp. 1834–1848, Sep. 2015.
- [25] M. Mueller, N. Smith, and B. Ghanem, "A benchmark and simulator for UAV tracking," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 445–461.
- [26] H. K. Galoogahi, A. Fagg, C. Huang, D. Ramanan, and S. Lucey, "Need for speed: A benchmark for higher frame rate object tracking," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1125–1134.
- [27] M. Kristan et al., "The sixth visual object tracking vot2018 challenge results," in *Proc. Eur. Conf. Comput. Vis. Workshops*, 2018.
- [28] J. Valmadre et al., "Long-term tracking in the wild: A benchmark," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 670–685.
- [29] X. Wang et al., "Towards more flexible and accurate object tracking with natural language: Algorithms and benchmark," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 13758–13768.
- [30] C. Mayer, M. Danelljan, D. P. Paudel, and L. Van Gool, "Learning target candidate association to keep track of what not to track," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 13424–13434.
- [31] B. Yan, H. Peng, J. Fu, D. Wang, and H. Lu, "Learning spatio-temporal transformer for visual tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10428–10437.
- [32] X. Chen, B. Yan, J. Zhu, D. Wang, X. Yang, and H. Lu, "Transformer tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8122–8131.
- [33] C. Mayer et al., "Transforming model prediction for tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 8721–8730.
- [34] S. Gao et al., "Aiatrack: Attention in attention for transformer visual tracking," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 146–164.
- [35] B. Chen et al., "Backbone is all your need: A simplified architecture for visual object tracking," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 375–392.
- [36] Y. Cui et al., "MixFormerV2: Efficient fully transformer tracking," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 58736–58751.
- [37] Y. Cui, C. Jiang, G. Wu, and L. Wang, "MixFormer: End-to-end tracking with iterative mixed attention," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 13608–13618.
- [38] X. Chen, H. Peng, D. Wang, H. Lu, and H. Hu, "SeqTrack: Sequence to sequence learning for visual object tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14572–14581.
- [39] S. Gao, C. Zhou, and J. Zhang, "Generalized relation modeling for transformer tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 18686–18695.
- [40] Y. Zheng et al., "ODTrack: Online dense temporal token learning for visual tracking," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 7588–7596.
- [41] C. Fu et al., "Rethinking general underwater object detection: Datasets, challenges, and solutions," *Neurocomputing*, vol. 517, pp. 243–256, 2023.
- [42] J. Ye, C. Fu, G. Zheng, D. P. Paudel, and G. Chen, "Unsupervised domain adaptation for nighttime aerial tracking," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 8886–8895.
- [43] Q. Yu, K. Fan, and Y. Zheng, "Domain adaptive transformer tracking under occlusions," *IEEE Trans. Multimedia*, vol. 25, pp. 1452–1461, 2023.



**Qiuyang Zhang** is currently working toward the M.S. degree in computer technology with the College of Information Technology, Shanghai Ocean University, Shanghai, China.

His research interests include deep learning, image processing, and object tracking.



**Wei Song** (Member, IEEE) received the Ph.D. degree in computer science from the Queensland University of Technology (QUT), Brisbane, QLD, Australia, in 2012.

From 2012 to 2015, she was a Research Fellow with QUT. She is currently a Professor with the College of Information Technology, Shanghai Ocean University, Shanghai, China. Her research interests include computer vision, remote sensing image analysis, time-series prediction, and their applications in marine fields.



**Cong Liu** received the bachelor's degree in software engineering from Shanghai Ocean University, Shanghai, China, in 2024.

His research interests include reinforcement learning, time-series forecasting, and image processing.



**Minghua Zhang** received the Ph.D. degree in computer science from Shanghai JiaoTong University, Shanghai, China, in 2017.

She became an Associate Professor with Shanghai Ocean University, Shanghai. Her research interests include underwater object detection, hyperspectral remote sensing image classification.